

Assignment 2: Eigen Values and Optimization

MECHTRON 3X03

Scientific Computation

Madeline LeBreton

Submitted November 21, 2025

Question 2: Deriving an analytical expression for eigenvalues and eigenvectors

To find the eigenvalues and eigenvectors of V , I investigated properties of the matrix. Using the matrix transpose, it can be shown that the matrix $V = v v^T$ is symmetric:

$$\begin{aligned} V^T &= (v v^T)^T \\ &= (v^T)^T v^T \\ &= v v^T \\ &= V \end{aligned}$$

Therefore, V is symmetric because $V^T = V$.

The effect of mapping on a vector $x \in R^n$ is investigated:

$$\begin{aligned} Vx &= (v v^T) x \\ Vx &= v (v^T x) \end{aligned}$$

Where $(v^T x)$ is a scalar, denoted c :

$$Vx = c v$$

So every vector Vx is a scalar multiple of v .

Given that $\|v\|_2 = 1$:

$$\begin{aligned} 1 &= \|v\|_2 \\ 1 &= \sqrt{(v^T v)} \end{aligned}$$

The first eigenvalue was found to be $\lambda = 1$ using the definition of V :

$$\begin{aligned} Vv &= (v v^T) v \\ Vv &= v (v^T v) \\ Vv &= 1 \cdot v \end{aligned}$$

Which fits the definition of a characteristic scaling $Vv = \lambda v$. Therefore, v is an eigenvector with $\lambda = 1$.

Symmetric matrices have eigenvectors orthogonal to each other:

$$v^T x = 0$$

$$Vx = v v^T x = v(0) = 0$$

Therefore, every vector orthogonal to v is an eigenvector with eigenvalue 0: $\lambda_2 = \lambda_3 = \dots = \lambda_n = 0$.

Question 4: Localizing a Receiver to n Transmitters

Using the general linear system approximation:

$$0 = F(x^*) \simeq F(x^{(k)}) + J(x^{(k)})(x^* - x^{(k)})$$

The Jacobian can be expressed as:

$$J(x) = D_x F$$

$$J(x^{(k)})_{ij} = \left. \frac{\partial F_i}{\partial x_j} \right|_{x=x^{(k)}}$$

Applying this to the given system:

$$f_1(x) = \|x - p_1\| - d_1 = 0$$

$$f_2(x) = \|x - p_2\| - d_2 = 0$$

$$\vdots$$

$$f_n(x) = \|x - p_n\| - d_n = 0,$$

To simplify, let $r_i = x - p_i$:

$$\begin{aligned} \frac{\partial \|r_i\|}{\partial x} &= \frac{\partial \sqrt{(r_i)^\top (r_i)}}{\partial x} \\ &= \frac{\partial (r_i^\top r_i)^{1/2}}{\partial x} \\ &= \frac{1}{2} (r_i^\top r_i)^{-1/2} \cdot 2r_i \\ &= \frac{r_i}{\|r_i\|} \end{aligned}$$

Using this in the partial derivative of F_i :

$$f_i(x) = \|r_i\| - d_i$$

$$\begin{aligned} \frac{\partial f_i}{\partial x_j} &= \frac{\partial (\|r_i\| - d_i)}{\partial x_j} \\ &= \frac{x - p_i}{\|x - p_i\|} \end{aligned}$$

The Jacobian can be written:

$$J(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} & \dots & \frac{\partial f}{\partial x_n} \end{bmatrix}$$

$$J(x) = \begin{bmatrix} \nabla f_1(x)^T \\ \nabla f_2(x)^T \\ \dots \\ \nabla f_n(x)^T \end{bmatrix} = \begin{bmatrix} \frac{(x-p_1)^T}{||x-p_1||} \\ \frac{(x-p_2)^T}{||x-p_2||} \\ \dots \\ \frac{(x-p_n)^T}{||x-p_n||} \end{bmatrix}$$

$$J_i(x) = \frac{(x-p_i)^T}{||x-p_i||}$$

Question 6: Nonlinear Least-Squares Optimization

Given the objective function where $f_i(x) = ||x - p_i|| - d_i$:

$$f(x) \triangleq \sum_{i=1}^m |f_i(x)|^2$$

The question is asking for the gradient ∇f with respect to x .

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \dots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

To simplify, again let $r_i = x - p_i$:

$$f_i(x) = \|r_i\| - d_i$$

$$\nabla_x f = \nabla \sum_{i=1}^m f_i(x)^2$$

$$\nabla_x f = \sum_{i=1}^m 2 f_i(x) \cdot \nabla f_i(x)$$

$$\nabla_x f = \sum_{i=1}^m 2(\|r_i\| - d_i) \cdot \nabla(\|r_i\| - d_i)$$

Using the finding from Q4 for $\frac{\partial \|r_i\|}{\partial x}$:

$$\nabla_x f = \sum_{i=1}^m 2(\|r_i\| - d_i) \cdot \left(-\frac{r_i}{\|r_i\|} + 0\right)$$

$$\nabla_x f = 2 \sum_{i=1}^m (\|r_i\| - d_i) \cdot \frac{r_i}{\|r_i\|}$$

Next, computing the Hessian of f with respect to x :

$$\nabla_x^2 f = \begin{bmatrix} \frac{\partial^2 f_1}{\partial^2 x_1} & \frac{\partial^2 f_1}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f_1}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f_2}{\partial x_2 \partial x_1} & \frac{\partial^2 f_2}{\partial^2 x_2} & \dots & \frac{\partial^2 f_2}{\partial x_2 \partial x_n} \\ \dots & \dots & \dots & \dots \\ \frac{\partial^2 f_n}{\partial x_n \partial x_1} & \frac{\partial^2 f_n}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f_n}{\partial^2 x_n} \end{bmatrix}$$

$$\nabla_x^2 f = \nabla_x (\nabla f)$$

To simplify, again let $r_i = x - p_i$, such that:

$$\nabla_x f = 2 \sum_{i=1}^m (\|r_i\| - d_i) \cdot \frac{r_i}{\|r_i\|}$$

Using the product rule:

$$\nabla_x^2 f = (2 \sum_{i=1}^m (\nabla f \nabla f^T + f \nabla^2 f))$$

$$\nabla_x^2 f = 2 \sum_{i=1}^m \left(\frac{r_i \cdot r_i^T}{\|r_i\|^2} - \frac{r_i - d}{\|r_i\|} I \right)$$

Question 7: Newton's Method Optimization Implementation

log10 of Cost

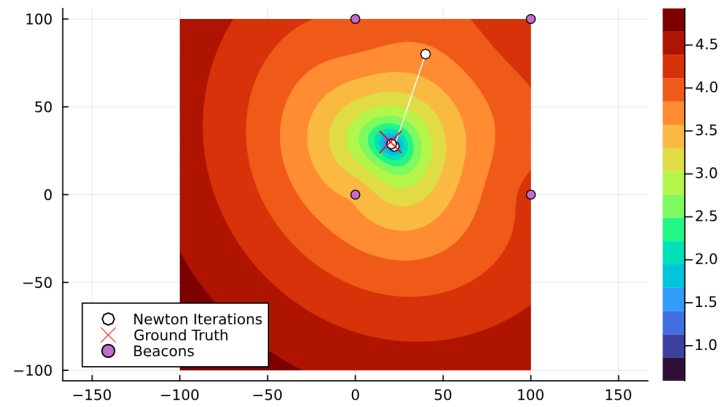


Figure 1: Plot of cost function for Newton's method-based optimizer

- a) To determine whether this implementation of Newton's method has converged, Julia code was added to numerically analyze the values of Δx and the gradient. The error tolerance was lowered to further examine the convergence behaviour.

```
println("\nIteration $iters \ngradient: $G \ndelta_x: $delta_x")
```

Figure 2: Julia code added to numerically analyze the convergence of Newton's method implementation.

```

Iteration 0
gradient: [63.39138389259833, 169.5315809798087]
delta_x: [18.053422121729064, 50.58356693782504]

Iteration 1
gradient: [9.75058844446665, 0.9428602714508867]
delta_x: [2.6498106153296295, -0.2307245132976898]

Iteration 2
gradient: [0.3417453835236284, -0.0969703885013935]
delta_x: [0.10221781302143795, -0.03970794911491531]

Iteration 3
gradient: [0.012049428726986555, -0.005784929871083477]
delta_x: [0.0037239198069771253, -0.001959588269083401]

Iteration 4
gradient: [0.00045435978709340574, -0.0002402034731103786]
delta_x: [0.0001415040893849167, -7.911986587590167e-5]

Iteration 5
gradient: [1.7435734588344687e-5, -9.369609855758299e-6]
delta_x: [5.4375429219975805e-6, -3.072193372920973e-6]

Iteration 6
gradient: [6.711982216689449e-7, -3.616672767936535e-7]
delta_x: [2.093689957255959e-7, -1.1849787317045517e-7]

Iteration 7
gradient: [2.585183889181053e-8, -1.3936115816903794e-8]
delta_x: [8.064346719896741e-9, -4.565517996927313e-9]

Iteration 8
gradient: [9.95794335878486e-10, -5.368985256382075e-10]
delta_x: [3.106372234074156e-10, -1.7588166221187532e-10]

Iteration 9
gradient: [3.832822947913428e-11, -2.067168658470564e-11]
delta_x: [1.1956772188726473e-11, -6.77122246804643e-12]

Iteration 10
gradient: [1.4948042803553108e-12, -7.94031507211912e-13]
delta_x: [4.657215156766817e-13, -2.611938353729614e-13]

Iteration 11
gradient: [5.395683899678261e-14, -1.2656542480726785e-14]
delta_x: [1.602984418409744e-14, -5.633519540925331e-15]

```

Figure 3: Code output showing numerical values for Δx and ∇f for each iteration of Newton's method.

These results show that the update step $\Delta x \rightarrow 0$, and therefore the method converges to a critical point since the values are no longer changing. If there were no error tolerance or maximum number of iterations, the step size would continue to approach 0, and the updated values would stop changing once it reaches a precision beyond ϵ_{mach} . The gradient also shrinks to zero, meaning that a local minimum or maximum has been reached because there is no single direction of increase. The necessary optimality condition for a function $f: R^n \rightarrow R$ is $\nabla f = 0$.

- b) The Hessian was analyzed to confirm whether this critical point is a local minimum or maximum. A critical point is a local minimum if the Hessian at the critical point is positive definite. To test this, the eigenvalues of the Hessian were evaluated.

```

println("\nH: $(H)")
println("eigenvalues of H: $(eigvals(H))")

```

Figure 4: Julia code added to display the eigenvalues of the Hessian matrix.

```
H: [3.7009324718862957 0.7504747954299663; 0.7504747954299663 4.299067528113703]
eigenvalues of H: [3.1921300816667997, 4.807869918333199]
```

Figure 5: Code output showing the eigenvalues of the Hessian matrix.

Since all of the eigenvalues of the Hessian are positive, it is a positive definite matrix as proven in lecture and therefore this critical point is a local minimum.

Question 8: Gradient Descent Method

b)

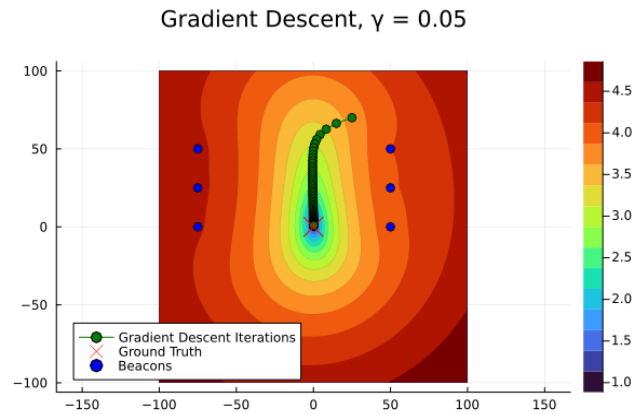


Figure 6: Plot of gradient descent on contour map with $\gamma = 0.05$

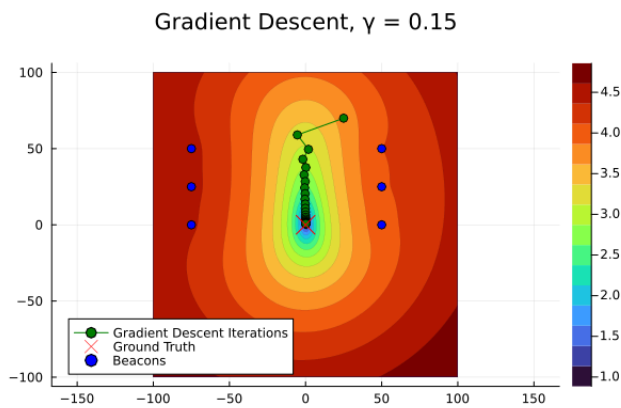


Figure 7: Plot of gradient descent on contour map with $\gamma = 0.15$

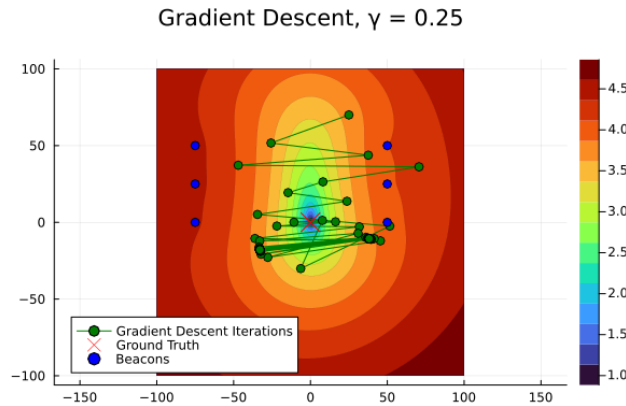


Figure 8: Plot of gradient descent on contour map with $\gamma = 0.25$

These plots show the effect of the step size γ on the convergence of gradient descent. For $\gamma = 0.05$, the gradient descent is observed to converge very slowly, moving in small steps towards the critical point. It requires many iterations and is computationally inefficient. For the moderate value $\gamma = 0.15$, the descent converges much faster, taking larger steps towards the critical point. This can be observed numerically from the code output, where $\gamma = 0.05$ required 121 iterations compared to the much faster $\gamma = 0.15$ with 37.

```
Gradient Descent iterations, γ=0.05: 121
Gradient Descent iterations, γ=0.15: 37
Gradient Descent iterations, γ=0.25: 201
```

Figure 9: Numerical output for the number of iterations at each value of γ

However, the effect of too large a step size is observed with $\gamma = 0.25$, where the descent overshoots the critical point and risks diverging depending on the application scenario. In this case, it does eventually converge to the critical point but requires 201 iterations. This method is also computationally inefficient due to the erratic behaviour.

Question 9

- a) Running the script several times, it can be observed that Newton's method doesn't always converge to the same value given the same starting conditions, while gradient descent generally does. This happens because the update step $\Delta x = H \setminus G$ depends on the inverse of the Hessian matrix. If H is near-singular, meaning that small changes in the input can cause large changes in output. This makes the updates unstable and very sensitive to noise, sending the iteration steps to unexpected values given the same input. A test that can be used to identify when the next step of Newton's method is more likely to behave erratically is checking the condition number of the Hessian:

$$\text{cond}(H) = \|H\| \cdot \|H^{-1}\|$$

If H is near-singular, it will have a very large condition number, indicating that the next step calculation is more likely to be affected by noise and overstep or converge to a different value.

- b) In general, it was observed that less iterations were required for Newton's method (7) than the gradient descent method (18-20).

```
Size of x_trace_n: (7,)
Size x_trace_g: (18,)
"/Users/madelinebreton/Documents/SCH00L/3X03/a2_q9_one_sol_with_iterations.png"

Size of x_trace_n: (7,)
Size x_trace_g: (19,)
"/Users/madelinebreton/Documents/SCH00L/3X03/a2_q9_one_sol_with_iterations.png"

Size of x_trace_n: (7,)
Size x_trace_g: (18,)
"/Users/madelinebreton/Documents/SCH00L/3X03/a2_q9_one_sol_with_iterations.png"

Size of x_trace_n: (7,)
Size x_trace_g: (20,)
"/Users/madelinebreton/Documents/SCH00L/3X03/a2_q9_one_sol_with_iterations.png"
```

Figure 10: Number of iterations required for each method over 4 iterations.

To understand why this is the case, the update functions are compared. The Newton's method update step $s^{(k)}$ is given by the equation:

$$\nabla^2 f(x^{(k)}) s^{(k)} = -\nabla f(x^{(k)})$$

And calculated in the step $\text{delta}_x = H \setminus G$. As proven in lecture, Newton's method has a quadratic convergence (order 2) in the neighbourhood of a root.

In contrast, the gradient descent update step:

$$x_{k+1} = x_k - \gamma \nabla f(x_k)$$

This step uses the first derivative at the current point to determine the iterative step, with a linear (first order) convergence. Therefore, it will converge slower than the quadratic convergence of Newton's method because it moves in the direction of steepest slope, but not necessarily towards the local minimum/maximum.